

AgentIA — Current State

Bare bones: how it functions, what ships, and what is honestly not verified

AgentIA Engineering

September 28, 2026

Scope of this document. The other three documents in `docs/` are SkillOpt-focused and were written *before* implementation: `notes.tex` is a proposal brief, `resume_skillopt.tex` is a method reference with prompt templates, and `slides.tex` is a 13-frame deck. None of them describes what is actually built. This document is that description, and it is deliberately blunt about the gaps.

Contents

1	What AgentIA is	2
2	How it functions	2
2.1	The LangGraph harness	2
2.2	The stage execution seam	3
2.3	The two generation modes	3
2.4	The compliance gate	3
2.5	The honest verifier (spec 012)	3
2.6	Cost tracing (spec 013)	4
2.7	The SkillOpt loop (spec 014)	4
3	Features shipped	4
4	The agent scripts	5
4.1	Generation and measurement	5
4.2	SkillOpt	5
5	Economics: where the numbers live	5
6	How to test it	6
6.1	Start both halves	6
6.2	One run that exercises the newest machinery	6
6.3	Two host-dependent behaviours to expect	7
7	Honest current limits	7
8	Next steps	7
8.1	Before anything else	7
8.2	SkillOpt design improvements	8
8.3	Documentation	8

1 What AgentIA is

AgentIA — *Microservice Code Studio* — ingests a formal specification of a microservice and produces a compiled, tested, exportable Spring Boot project. It is two layers that talk over REST and Server-Sent Events:

React / Vite Studio (port 3000)	FastAPI Orchestrator (port 8000)
Spec ingestion (BDD, architecture, relational models)	FIFO concurrency queue (max 2 simultaneous builds)
Live generation monitor over SSE	LangGraph autonomous agent (8 nodes)
Code explorer, security audit, DevOps, export	Hermetic Docker sandbox (<code>mvn test -o -network none</code>)
ZIP download and atomic Git publish	Bounded self-repair loop
	SQLite session store

The property that makes it more than a code generator is that **it refuses to claim success it did not verify**. That constraint was not present at the start; it was built in specs 012 and 013, and it is the most important thing to understand about the current state.

2 How it functions

2.1 The LangGraph harness

The agent is an eight-node graph. Topology as it exists in `backend/app/orchestrator/graph.py`:

```
START -> validator
validator --(FAILED)--> END
          --(else)----> scaffolder
scaffolder -> domain -> service -> controller -> test -> sandbox
sandbox    --(build_success)--> END
          --(status BLOCKED)--> END
          --(else)-----> repair
repair     --(status BLOCKED)--> END
          --(else)-----> sandbox
```

Node	Responsibility
validator	Reject an unusable blueprint before any model call
scaffolder	pom.xml, application.yml, @SpringBootApplication
domain	JPA entities and DTO records
service	Service interfaces and implementations, repositories, exceptions
controller	REST controllers; error handling delegated to one advice class
test	One test per declared acceptance scenario
sandbox	Hermetic offline build and test, then the honesty check
repair	Bounded self-repair from parsed Maven output

Two routing details matter and are easy to miss:

- `sandbox -> END` on `BLOCKED`. An unverifiable workspace does **not** enter the repair loop: no code patch fixes a missing container runtime. This is what makes spec 012's honest failure terminate cleanly rather than burning the repair budget on an environment fault.
- `repair -> sandbox` is the loop, and it is bounded by the configured repair cap.

2.2 The stage execution seam

Every generation stage — in both execution paths — is invoked through one function, `stages/runner.py::run_stage`. It is the only place that owns mode selection, request and correction budgeting, compliance gating, provenance recording, cost recording, and journal accumulation. That the two paths share it is what makes behavioural parity structural rather than a matter of discipline.

Path	Entry
A — graph	<code>generation_graph.stream</code> via <code>routes_session.execute_generation_pipeline</code> ; nodes are thin delegations into the seam
B — sequential	<code>pipeline_runner._execute_pipeline_steps</code> (auto-pilot), calling the same seam directly

2.3 The two generation modes

A session is decided to be `MODEL` or `DETERMINISTIC` once, at start, and never changes. `MODEL` never falls back to deterministic.

	DETERMINISTIC	MODEL
Implementation	<code>stages/deterministic/*.emit</code>	<code>stages/model/*</code>
Compliance gate	none	applied before persistence
Model requests	0	1 initial + up to 2 corrections per stage
Provenance	no provider or model recorded	provider, model, revision, ordinal

The gate asymmetry is deliberate (FR-013): deterministic output is compliant by construction, and gating it would change offline behaviour. It is documented at the point of use so it is not later “fixed” as an oversight.

2.4 The compliance gate

Two independently-written validator families evaluate every candidate artifact set. They overlap and **disagree on severity** for the same violation — Lombok `@Data` on a JPA entity is `MEDIUM` in one and `HIGH` in the other. The adapter keeps the strictest, and that tightening is a real behaviour change: it can convert previously-completing sessions into blocked ones. It is recorded in `specs/011-llm-generation-nodes/migration-notes.md` because it is the single most likely source of a false regression attribution.

Violations are attributed one of two ways, and the distinction is load-bearing:

Attribution	Effect	Canonical case
LOCAL	Rejects the candidate set; consumes a correction attempt	DTO declared as a class
ACCUMULATED	Recorded, never rejects, never consumes an attempt	No <code>@RestControllerAdvice</code> annotation

The accumulated case is why a scaffolder running *before* the controller stage is not rejected for a missing global error handler it could not have written.

2.5 The honest verifier (spec 012)

The sandbox originally substituted a synthetic `BUILD SUCCESS` with fabricated 5/5 test counts whenever it could not build. Sessions therefore reached `VERIFIED` for workspaces that were never compiled, and every downstream figure inherited that. It is now:

Outcome	exit_code	fallback_used
Verified and passed	0	false
Verified and failed	non-zero	false
Could not verify	non-zero	true

There are **four** substitution triggers, not the three originally identified: runtime unavailable, runtime executable missing, runtime communication failure, and — the most severe — a build failure whose output matches an environment-looking pattern, which used to be converted into a *pass*.

An unverifiable session terminates in **BLOCKED**, bypassing repair, with the reason recorded. **ALLOW_HERMETIC_FALLBACK** (default **false**) restores the old behaviour for local development; the marking is recorded in **both** modes, because permissive mode changes what is permitted, not what is recorded.

2.6 Cost tracing (spec 013)

Every model call is recorded through a proxy applied by the model factory — but only while a recording context is active, which is true inside the stage boundary and false for a direct factory call. That condition keys on *is recording active*, never on *is this a test*; a test-shaped bypass would leave production unrecorded while the suite stayed green.

The local SQLite store is the **system of record**. The telemetry destination is a mirror whose absence is a non-event, so the report is deterministic and offline. Pricing is per call, not per model, because the provider’s published rates vary by peak/off-peak (2×) and by cache hit/miss (up to 100× on input).

2.7 The SkillOpt loop (spec 014)

One iteration, one skill, no memory between runs:

```
collect recorded outcomes -> reflect into <=4 edits -> apply to a COPY
-> gate BOTH skills on the SAME fresh executions
-> accept iff candidate_score > current_score (strict; ties rejected)
-> log exactly one run record
```

Every stability mechanism the source method uses is deliberately absent: no rejected-edit buffer, no learning-rate schedule, no epoch-wise slow/meta update, no multi-skill bank, no significance test. The protected **SLOW_UPDATE** region is already honoured so that adding the slow update later needs no change to the edit contract.

3 Features shipped

Spec	Feature	State
001	Microservice Code Studio	Core platform: ingestion, queue, session store, SSE
002	Requirements to stories	BDD ingestion to Given/When/Then
003	Architecture component design	Component and Mermaid rendering
004	Domain models / SQL	Relational model generation
005	Automated tests analysis	Test suite analysis and repair histories
006	Security & quality audit	SAST, quality gate, export blocking
007	Docker CI/CD orchestration	Deployment and pipeline surfaces
008	Unified workflow orchestration	Lifecycle, phases, auto-pilot
009	Historical baseline analysis	Synthetic incident baseline

Spec	Feature	State
010	Skill injection	Domain triage only — the skill documents were never authored
011	LLM generation nodes	Built. Seam, 5 model stages, gate, correction loop, provenance, journal, measurement harness
012	Sandbox verifier honesty	Built. Four triggers, permissive opt-in, terminal-state parity
013	Cost tracing	Built. Per-call telemetry, aggregation, report, pricing table
014	SkillOpt compact	Built. One-iteration loop, strict gate, disjoint evidence

Spec 010 is the notable gap. It classified the failure domains and created three skill files, but all three are **zero bytes** — placeholders. Only `layer_architecture.md` has content today, authored as feature 014’s seed. `exception_handling.md` and `mockito_tests.md` remain empty.

4 The agent scripts

Everything under `backend/scripts/` is run directly; none is wired into the API.

4.1 Generation and measurement

Script	Purpose
<code>capture_generation_baseline.py</code>	Freezes the pre-migration deterministic output (spec 011 T008). Must run <i>before</i> any stage is migrated.
<code>analyze_historical_baseline.py</code>	Analyses the synthetic incident baseline (spec 009)
<code>seed_historical_baseline.py</code>	Seeds it
<code>report_session_costs.py</code>	The cost report. Prints the headline average cost per completed session with its population, exclusions and pricing basis
<code>validate_seed_skills.py</code>	Validates skill documents

4.2 SkillOpt

Module	Role
<code>run_skillopt.py</code>	The orchestrator: one iteration, one run record
<code>skillopt/collect.py</code>	Recorded sessions to outcome records. “No failures” $\neq$ “no sessions”; a fallback-marked session counts as a failure
<code>skillopt/reflect.py</code>	One direct factory call to a bounded edit set
<code>skillopt/apply.py</code>	Four operations, cap $L_t = 4$, applied to a copy ; protected-region and not-found targets rejected individually
<code>skillopt/gate.py</code>	Both skills on the same fresh executions; strict acceptance; disjointness asserted in code
<code>skillopt/real_runner.py</code>	The production session runner, used only by <code>-real</code>
<code>skillopt/prompts/analyst_err</code>	The reflector prompt, adapted from SkillOpt Appendix C.2.1

5 Economics: where the numbers live

What	Where
The headline report	backend/scripts/report_session_costs.py
System of record	backend/cost_tracking.db — tables model_calls and session_costs
Published rate table	backend/app/resources/model_pricing.json (cites its source URL in-file)
Intervention rate	reports/measurements/011-sc011-intervention-rate.md
Excluded-session count	reports/measurements/012-fallback-excluded-sessions.md
Frozen pre-migration baseline	reports/baselines/ — immutable

Producing the number:

```
PYTHONPATH=backend .venv/bin/python backend/scripts/report_session_costs.py
```

There is no cost figure yet, and reporting one would be the failure this platform was built to prevent. The cost store is currently **empty**: no session has run with recording active. The report says so explicitly — *“No sessions found... This is not a measurement of zero cost”* — and that refusal is the correct behaviour, not a missing feature.

The only quantified figure that exists is the SC-011 intervention rate: **12.5% over 32 sessions**, and it is labelled **synthetic** in the artifact itself, because those sessions were driven by a fake client with a scripted compliant/non-compliant mix. It measures the mechanism, not a real model.

6 How to test it

6.1 Start both halves

The .bat and .ps1 launchers assume Windows and a bare python. On Linux:

```
# backend -> http://localhost:8000 (Swagger UI at /docs)
cd backend && ../.venv/bin/python -m uvicorn app.main:app --reload --port 8000

# frontend -> http://localhost:3000
cd frontend && npm run dev
```

frontend/src/services/apiClient.ts uses baseUrl: '/api/v1', so the Vite dev server needs a proxy to port 8000.

6.2 One run that exercises the newest machinery

A single MODEL-mode session on the minimal blueprint touches the stage seam (011), the honest verifier (012) and cost recording (013).

#	View	What to observe
1	SpecIngestionView	Paste backend/tests/fixtures/baseline_blueprints/minimal.json; provider deepseek + key
2	GenerationMonitorView	Live SSE: phase transitions, build logs, and the verification_result event carrying verificationFallbackUsed
3	CodeExplorerView	Generated artifacts and their provenance
4	ExportPublishView	The ZIP. A 403 means the quality gate blocked the export — correct, not a bug

API equivalent (request shape verified against `QuickStartSessionRequest`):

```
curl -X POST http://localhost:8000/api/v1/sessions/quick-start \
  -H 'Content-Type: application/json' \
  -d '{"serviceName":"notes-service","rawText":"<minimal.json>","autoRun":true,"llmProvider":"deepseek","apiKey":"sk-..."}'

# then:
# GET /api/v1/sessions/{id}/stream      SSE
# GET /api/v1/sessions/{id}             detail (+ verificationFallbackUsed)
# GET /api/v1/sessions/{id}/artifacts   listing
```

6.3 Two host-dependent behaviours to expect

The sandbox result depends on container-runtime reachability. If the runtime is unreachable and `ALLOW_HERMETIC_FALLBACK` is unset, the session **blocks**. That is spec 012 working. To run the loop end to end on such a host, either enable the permissive flag deliberately or make the runtime reachable.

The SkillOpt real iteration needs two flags, not one. A key in `backend/.env` reaches `os.environ` through `load_dotenv()`, so a key-presence check passes without the operator realising a key exists. It therefore requires `AGENTIA_RUN_REAL_SKILLOPT=1` *and* `AGENTIA_ALLOW_REAL_SPEND=1`, and it spends real money.

7 Honest current limits

Recorded here rather than left to be discovered.

Limit	Detail
No cost figure exists	The store is empty. Running the report is the correct first step; reporting a number before that is the failure mode 012/013 close.
SC-003 not verified by the implementer	The real-build criterion was verified by the operator from a capable shell; the implementer's shell cannot reach the container runtime. Recorded as such, never inferred.
SC-007 partially observed	A real SkillOpt iteration ran twice unintentionally and reached a decision, but its scores were suppressed by the test and were not re-run, to avoid spending again without authorisation.
The gate is coarse	Five blueprints, $M = 4$: one task moves the pass rate by 25 points . Noise can decide an acceptance.
No memory between runs	A rejected edit is logged and dropped, so v1 can repeat a failed edit indefinitely and nothing notices.
Repair-cap drift	The constitution specifies 3; <code>config.py</code> sets 5 . This flatters any absolute intervention-rate ceiling and is disclosed in the measurement artifact.
Three skill files empty	<code>exception_handling.md</code> and <code>mockito_tests.md</code> are zero bytes; only the seed has content.
No frontend surface for cost	The report is a CLI artifact. There is no view for it.

8 Next steps

8.1 Before anything else

Widen the held-out exam. Adding optimizer machinery to a five-task gate optimises against noise. This is the highest-value change and it is cheap.

Run sessions with recording on. The economics are unmeasurable until the store has data, and every other cost question is downstream of that.

8.2 SkillOpt design improvements

1. **Filter environment failures in code, not in the prompt.** The collector currently classifies fallback-marked sessions as failures, sends them to the reflector, and the prompt then asks the model to ignore them. Keeping them for context is defensible, but the ignore-instruction should not be the only guard.
2. **Cache the current score** keyed by (skill hash, held-out set). The gate is $2 \times M$ real sessions per iteration; caching makes an epoch loop affordable at all.
3. **Record a real build exit code.** It is currently *derived* from the verification metrics' pass flag because the session table has no such column.
4. **Pass provider and key into the session state** in `real_runner`; a real run currently depends on ambient environment detection.
5. **Then, in the source method's order:** rejected-edit buffer $\rightarrow$ textual learning-rate schedule $\rightarrow$ epoch/slow-meta update $\rightarrow$ multi-skill bank. The protected region already exists for the slow update.
6. **Statistical validation** — the deferred item that makes any of the above trustworthy at this sample size.

8.3 Documentation

`README.md` predates specs 011–014 entirely: it still describes the agent as blocks that no longer exist, and states a repair cap of 3 when the code sets 5. It should either point here or be brought current.